

React –

It is a Front-end framework developed by Facebook in 2011.

Npx (Node Package Execute) is a command that allows to use js libraries without installing it locally.

```
npm install react  
npx create-react-app [app_name]
```

Types of view -

Declarative view -> React updates and render component when data changes.

Component Based -> Separate component for each element, which then can be composed to make complex UI

Working-

Virtual DOM created which is exact replica of real DOM.

React updates the corresponding nodes in Virtual DOM and compares it with real DOM. Upon comparing, changes in nodes are made in real DOM and is reflected on browser. This comparing of real DOM and virtual DOM is done by React-DOM

To create React app using Vite, following command -

```
npm create vite@latest
```

Types of Export -

Named Export -> It is used to export class

Syntax ->

```
export class ABC() {  
    console.log("Hello World");  
}
```

Default Export -> It is used to export class

Syntax ->

```
class ABC() {  
    console.log("Hello World");  
}
```

JSX (JavaScript Syntax Extension)-

React uses this language.

It is a mixture of JS and html which helps to write html and JavaScript together.

In JSX, every tag should have one parent element.

To overcome the problem of having parent element -

<React.Fragment> element used to return multiple child elements without adding extra node to DOM.

In other words, it is used to return multiple child elements without having parent element

Syntax ->

```
<React.Fragment> {piece of code} </React.Fragment>
```

Component-

These are re-usable bits of code.

Returns Html

Example of Component -> Navbar.js, ProductList.js

Types of Components -

- Class Based
- Function Based

To start Development Server, Following command –

npm install
npm run dev

Simple React Code of a Component –

import React from "react"

```
class Navbar extends React.Component {  
  return() {  
    render(  
      <div className = "Hero">  
        <p>This is my div</p>  
      </div>  
    )  
  }  
}
```

Normal Flow of React app-

Lg.jsx file

```
function ListGrp() {  
  return (  

```

```

    <>
    <h1>This is Fruits List</h1>
    <ul>
      <li>Apple</li>
      <li>Orange</li>
    </ul>
    </>
  )
}

```

```
export default ListGrp;
```

```

    |
    |
    |
    v

```

```

import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import ListGrp from '../components/LG.jsx'

```

```

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <ListGrp />
  </StrictMode>,
)

```

Note –

To use JavaScript inside render keyword, we write JavaScript code inside curly braces {}.

To use variables, we write it in curly braces to access the variables. E.g- {item}

Use State –

It is a Hook in react js which is a built in react utility that contains data that gets changed over time

Syntax -> `const [variable_name , updater_function] = [value_of_variable]`

Props-

They are inputs to the component. They can be achieved by interfaces.

Passing Data Via Props-

E.g. -

interface [name_of_prop]:

 Name: String
 Heading: String

To use this variable –

Function Intro ({Name, Heading}: ABC)

```
    return (  
      <h1>{Name}</h1>  
      <h2>[Heading]  
    )  
  }
```

Export default Intro

Pass the value for the variable inside app.tsx -

```
Function App() {  
    <Intro name = "Prateek" /> }  
}
```

Passing Functions as Props-

E.g. -

```
interface [name_of_prop]:  
    list_item: string[];  
    onSelect: (item: string) => void;
```

To use this function –

```
Function prep(list_item , onSelect): pr {  
    onClick={() => onSelect(item)}  
}
```

Export default prep;

Pass the function in app.tsx –

Let item = ["Hello" , "World"]

```
<Prep onSelect={{item} => console.log(item)}
```

React Fc -

It is a function Component in react that provides type safety to props that the components expects to receive

```
Syntax – const [component_name]:ReactFC<prop_name> = ({prop_name1 , prop_name2}) => {  
    return (  
        {prop_name1}  
    )  
}
```

Eg – const MyComponent:ReactFc<AppProps> = ({name , age}) => {

```
    return (  
        {name} , {age}  
    )  
}
```

Children –

It is a special prop that helps to pass the parent component to child component. In simple words, it is used to pass the content as the children of parent component.

E.g -

Component.tsx -

```
Interface appProp {  
    Children : React.ReactNode  
}
```

Pass it in function -

```
function MyComponent:React.FC<appProp> = () => {  
    return (  

```

```
        <div>
          {children}
        </div>
      )
    }
  }
```

Export default MyComponent

Pass the content as children in the component –

```
<MyComponent>
  <h1>Hello</h1>
  <h2>World</h2>
</MyComponent>
```

To use named export -

import {Function name} from path

Note – function name should be enclosed in curly braces always in case of using named export

Prop Drilling -

When goes level by level to reach actual target, that concept is called Prop Drilling.

For e.g.-

If one prop has to flow from parent to its child components.

Parent -> C1 -> C2 -> C3

useContext –

It is a react utility that eliminates the problem of Prop Drilling. It works by following a three-stage process –

Create the Context .To create context -

write -> `const {variable} = createContext()`

Provide the Context by parent –

`<{context_name}.Provider value={value_name}>`

Name of the Consumer -> `<{Child4} />`

`<{context_name}.Provider>`

Don't forget to export the component -

`export {Context_name}`

Consume the Context by any of child component. We write -

`<{Context_name}. value>`

useReducer –

It is an react js hook which is an advanced version of useState hook used to change the variable based on action type

Syntax -> `const [state, dispatch] = useReducer(reducer, initialState);`

- reducer: A function that determines how the state should change.
- initialState: The initial value of the state.
- state: Current state.
- dispatch: A function to send actions to the reducer.